

Conversational SHL Assessment Recommender

Approach Summary for the SHL AI Intern Assignment

Problem Framing

The assignment is not a simple retrieval task. The service must act like a narrow conversational agent over the SHL catalog, remain fully stateless, return a strict JSON schema, and behave correctly across clarification, recommendation, refinement, comparison, and refusal. My design goal was therefore to make the *selection layer deterministic and auditable*, while still using modern retrieval and optional LLM support where they add value.

The final system is a FastAPI service with two endpoints: `GET /health` and `POST /chat`. Every `/chat` request carries the full conversation history. The server stores no session state. Instead, it reconstructs the user's current intent and constraints from the incoming messages on every call, then routes the conversation to one of five intents: `clarify`, `recommend`, `refine`, `compare`, or `refuse`.

System Architecture

The runtime path is intentionally compact:

1. Load the local SHL catalog from JSON.
2. Validate request shape with Pydantic.
3. Rebuild conversational state from the full message history.
4. Run safety and scope checks first.
5. Optionally enrich the extracted need with Groq or Gemini JSON hints.
6. Retrieve and rank only catalog items.
7. Bind recommendations back to exact catalog rows before responding.

This separation is important. The model can help interpret the conversation, but it never chooses arbitrary URLs or invents assessment rows. The final recommendation objects always come from the local catalog and are validated before the response is returned.

Catalog Pipeline and Grounding

I keep catalog collection offline rather than scraping at request time. The scraper under `scripts/` crawls SHL's product catalog and stores a local JSON dataset. At runtime, the service loads that file, normalizes product names, repairs malformed JSON control characters if needed, maps SHL category labels to the compact `test_type` codes required by the assignment, and filters the catalog to *Individual Test Solutions*.

Because the public catalog can contain noisy rows, I added conservative scope heuristics and canonicalization. Examples include preserving valid multi-type individual assessments such as `Global Skills Assessment`, normalizing broken product names such as `Microsoft Excel 365 (New)`, deduplicating by URL, and rejecting job-solution bundles. This matters because one hallucinated or out-of-scope recommendation would fail the hard evaluator checks.

Conversational State Reconstruction

The stateless nature of the API is central to the design. Instead of relying on a server-side conversation object, the system recomputes a structured `NeedState` from the full transcript. That state includes:

- inferred role title and seniority,
- salient skills and domain terms,
- desired assessment types such as P, A, K, B, or S,
- duration, language, and remote preferences,
- whether the user is comparing two named products,
- whether the conversation appears complete.

The heuristics are deliberately tuned for the assignment constraints. The evaluator caps conversations at 8 total messages, so the agent uses a low threshold for "enough context" and can force a best-effort recommendation near the turn limit instead of over-clarifying. This improves practical task completion without breaking the clarification requirement for truly vague first turns.

Retrieval and Ranking

The recommender is hybrid rather than purely semantic or purely rule-based. The first layer is a deterministic lexical scorer over the full catalog. It uses token overlap, bag-of-words match, job-level fit, duration fit, language fit, remote fit, and requested test-type match. I also added penalties to avoid low-value outputs such as report-heavy lists when the user is clearly asking for assessments.

The second layer is optional local semantic retrieval with a precomputed `sentence-transformers` embedding index. Query embeddings are compared against catalog embeddings, and semantic scores are fused with lexical scores using tunable weights. This helps the agent handle fuzzy or JD-style queries while keeping the final ranking stable and fast.

On top of this baseline, I added a thin layer of catalog-grounded anchors for recurring public-trace patterns such as Rust infrastructure hiring, senior leadership selection, contact-center screening, graduate finance hiring, sales re-skilling, healthcare admin, and Java full-stack roles. These anchors are not free-form rules that fabricate items; they simply promote exact catalog rows that are known to be especially relevant for those query families. This was the most effective way to improve trace-level recall while remaining grounded.

Behavior Design and Safety

The response layer is built around the assignment's required behaviors.

- **Clarify:** vague inputs produce one focused follow-up question and no recommendations.
- **Recommend:** once enough context is present, the service returns 1–10 catalog-bound recommendations.

- **Refine:** new constraints such as “add personality”, “drop REST”, or “remove OPQ” are applied to the latest shortlist without resetting the whole conversation.
- **Compare:** product names are resolved with exact match, aliases, substring match, and fuzzy matching, then compared only through catalog fields.
- **Refuse:** prompt injection, cheating, legal questions, and general hiring advice are blocked before retrieval.

I treated refusal and schema reliability as first-class engineering problems. The API always returns the assignment JSON shape, even on malformed requests, timeouts, or internal exceptions. This is enforced by response models plus explicit exception handlers for /chat.

Optional LLM Assistance

The system can optionally call Groq or Gemini, but only for *structured hints*. The prompt forces JSON output containing retrieval phrasing, additional skills, seniority cues, duration hints, and desired assessment types. It explicitly forbids product names and URLs. If the model times out, rate-limits, or returns invalid JSON, the service falls back to the deterministic rule-based state extractor. This gives the system the flexibility of an LLM without letting it become the source of truth.

Optional Visuals

If the report layout permits, the following four figure slots can be populated by renaming your generated images to the listed filenames under a `figures/` folder.

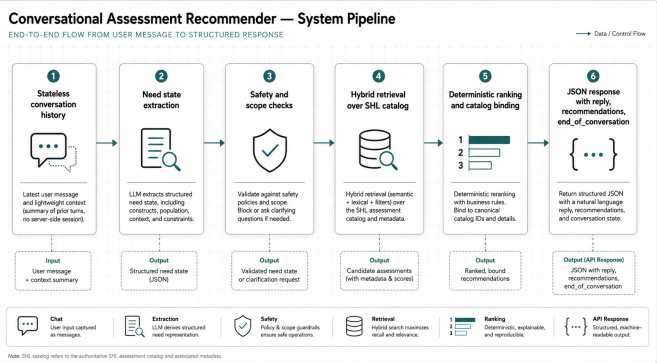


Figure 1: Stateless request flow from transcript to grounded shortlist.

Latency, Deployment, and Evaluation

To stay within the evaluator’s timeout budget, the service warms the embedding stack at startup and applies a wall-clock budget to /chat. The catalog is always local, the embedding index is optional but precomputed, and the API has no conversation-level storage, which makes deployment simple on platforms such as Render.

Evaluation is driven by tests rather than manual spot checking alone. The codebase includes contract tests for schema compliance and behavior probes for vague-query clarification, prompt injection refusal, comparison quality, refinement handling, turn-budget behavior, scope filtering, and several public-trace style scenarios. This let me improve recall without regressing hard constraints.

Trade-offs and Final Positioning

I intentionally did not build a free-form chatbot that “sounds smart” but can hallucinate SHL products. Instead, I chose a retrieval-first, catalog-bound system that is easier to defend in interview and safer under automated evaluation. The main limitation is that some trace-specific improvements are driven by catalog-aware ranking heuristics rather than a learned reranker. I consider that a reasonable trade-off for an assignment where correctness, grounding, and schema reliability matter more than model sophistication.

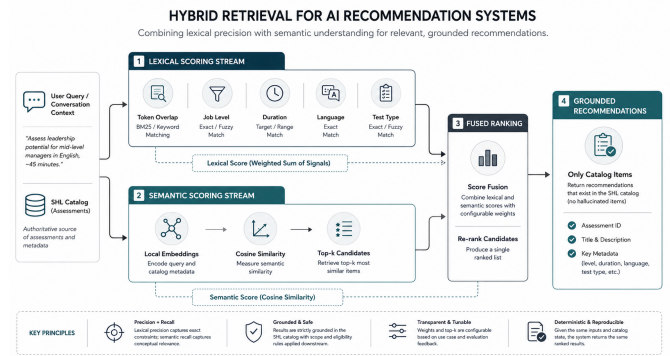


Figure 2: Lexical and semantic retrieval streams fused before final ranking.